

Step 2 — One Tool, One Loop: the Smallest Possible Working Agent — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the loop, proven once

```
# tools.py
from langchain_core.tools import tool

@tool
def calculator(expression: str) -> str:
    """Evaluate a basic arithmetic expression, like '47 * 6'."""
    return str(eval(expression))
```

```
# agent.py
from langchain_core.messages import SystemMessage, HumanMessage, ToolMessage
from langchain_openai import ChatOpenAI
from tools import calculator

def run_agent(task):
    model = ChatOpenAI(model="gpt-4o-mini").bind_tools([calculator])
    messages = [SystemMessage("Use the calculator tool for any math."), HumanMessage(task)]

    while True:
        response = model.invoke(messages)
        messages.append(response)
        if not response.tool_calls:
            return response.content
        for call in response.tool_calls:
            result = calculator.invoke(call["args"])
            messages.append(ToolMessage(content=result, tool_call_id=call["id"]))
```

```
# main.py
from agent import run_agent
```

```
print(run_agent("What is 47 * 6?"))  
print(run_agent("What's the capital of Japan?"))
```

Expected output:

```
47 * 6 is 282.  
The capital of Japan is Tokyo.
```

The first prompt makes exactly one tool-call round before answering; the second never calls the tool at all — same model, same loop, the decision to use the tool or not is the model's, based on the task. This works, but `calculator` has no argument schema (the model is trusted to send a valid `expression` string with nothing checking it), and `run_agent()` returns a bare string with no record of what actually happened during the run.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a Pydantic argument schema and a structured `AgentResult`

```
# tools.py
from langchain_core.tools import tool
from pydantic import BaseModel

class CalculatorArgs(BaseModel):
    expression: str

@tool(args_schema=CalculatorArgs)
def calculator(expression: str) -> str:
    """Evaluate a basic arithmetic expression, like '47 * 6'."""
    return str(eval(expression))
```

```
# agent.py
from pydantic import BaseModel
from langchain_core.messages import SystemMessage, HumanMessage, ToolMessage
from langchain_openai import ChatOpenAI
from tools import calculator

class AgentResult(BaseModel):
    answer: str
    steps: list[str]

def run_agent(task: str) -> AgentResult:
    model = ChatOpenAI(model="gpt-4o-mini").bind_tools([calculator])
    messages = [SystemMessage("Use the calculator tool for any math."), HumanMessage(task)]
    steps: list[str] = []

    while True:
        response = model.invoke(messages)
        messages.append(response)

        if not response.tool_calls:
```

```

        return AgentResult(answer=response.content, steps=steps)

    for call in response.tool_calls:
        result = calculator.invoke(call["args"])
        steps.append(f"called {call['name']} with {call['args']} -> {result}")
        messages.append(ToolMessage(content=result, tool_call_id=call["id"]))

```

```

# main.py
from agent import run_agent

for task in ["What is 47 * 6?", "What's the capital of Japan?"]:
    result = run_agent(task)
    print(f"Task: {task}")
    for step in result.steps:
        print(f"    {step}")
    print(f"Answer: {result.answer}\n")

```

Expected output:

```

Task: What is 47 * 6?
    called calculator with {'expression': '47 * 6'} -> 282
Answer: 47 * 6 is 282.

Task: What's the capital of Japan?
Answer: The capital of Japan is Tokyo.

```

Notice the second task's `steps` list is empty — proof the loop only calls the tool when it actually needs to, not because it was told to.

Difference from Basic: `CalculatorArgs` gives LangChain (and the model) a real schema for the tool's one argument, and `AgentResult` turns "what happened during this run" into something you can iterate over and test, instead of a string you'd have to re-parse. The loop's logic hasn't changed at all — same ask → check → run-or-stop shape as Basic.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a failing tool call that doesn't crash the loop

```
# agent.py
from pydantic import BaseModel
from langchain_core.messages import SystemMessage, HumanMessage, ToolMessage
from langchain_openai import ChatOpenAI
from tools import calculator

class AgentStep(BaseModel):
    tool: str
    args: dict
    result: str
    ok: bool

class AgentResult(BaseModel):
    answer: str
    steps: list[AgentStep]

def run_agent(task: str) -> AgentResult:
    model = ChatOpenAI(model="gpt-4o-mini").bind_tools([calculator])
    messages = [SystemMessage("Use the calculator tool for any math."), HumanMessage(task)]
    steps: list[AgentStep] = []

    while True:
        response = model.invoke(messages)
        messages.append(response)

        if not response.tool_calls:
            return AgentResult(answer=response.content, steps=steps)

        for call in response.tool_calls:
            try:
                result = calculator.invoke(call["args"])
                ok = True
            except Exception as e:
                result = f"Error: {e}"
```

```

        ok = False

        steps.append(AgentStep(tool=call["name"], args=call["args"], result=result))
        messages.append(ToolMessage(content=result, tool_call_id=call["id"]))

```

Expected output, run against a task engineered to send a bad expression (like "What is 47 ** ** 6?", which `eval` can't parse):

```

Task: What is 47 ** ** 6?
  calculator({'expression': '47 ** ** 6'}) -> Error: invalid syntax (<string>, li
  calculator({'expression': '47 ** 6'}) -> 10779215329 [ok=True]
Answer: 47 ** 6 is 10,779,215,329.

```

The exact retry behavior depends on the model, but the pattern holds: the first `AgentStep` has `ok=False` and a labeled error instead of a crash, the model sees that error as a normal `ToolMessage`, and — because the error is clear and machine-readable, not a stack trace — it often corrects its own syntax and tries again, arriving at a real answer instead of the whole run dying on the first bad call.

Approach 2 — same loop, printed as a readable think/act/observe log

```

# main.py
from agent import run_agent

result = run_agent("What is 47 * 6?")
for i, step in enumerate(result.steps, start=1):
    status = "ok" if step.ok else "FAILED"
    print(f"Step {i} [{status}]: {step.tool}({step.args}) -> {step.result}")
print(f"Final answer: {result.answer}")

```

Expected output:

```

Step 1 [ok]: calculator({'expression': '47 * 6'}) -> 282
Final answer: 47 * 6 is 282.

```

This is the exact log format the README asks `main.py` to print, and it's the same shape Step 3 extends with a step count against `max_iterations` — nothing new gets invented there, just a limit check added around this same per-step loop.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's loop assumes every tool call succeeds — true almost always for `calculator`, but not something you can assume in general. Approach 1 adds the `try/except` and the `ok` flag so a failure is visible data instead of a crash, which is the exact seam Step 3's genuinely-failing tool plugs into. Approach

2 doesn't change the agent at all — it's just `main.py` turning the same `AgentStep` list into the human-readable log the README's Step 2 deliverable asks for.

Which one should you actually write? The full Advanced version — the `try/except`, the `AgentStep` model, and the readable log — because Step 3 reuses this loop's shape wholesale, just with more tools and a `max_iterations` wrapped around the same `while True:`. Writing the failure path now, against a tool simple enough that failures are easy to engineer on purpose, means Step 3's actually-flaky tool has somewhere correct to plug into instead of needing this same fix invented under more pressure.